

SUPSI

Proxy basato su Appwrite

Studente/i

**Mazza Luca
Mondini Manuela**

Relatore

Brocco Amos

Correlatore

-

Committente

Brocco Amos

Corso di laurea

**Ingegneria informatica
(Informatica TP)**

Modulo

C11160

Anno

2025 / 2026

Data

25 febbraio 2026

Indice

1	Abstract	1
2	Introduzione	2
2.1	Pianificazione	3
2.2	Requisiti	3
3	Stato dell'arte	4
4	Design e implementazione	5
4.1	API	5
5	Test e validazione	6
5.1	Risultati	6
6	Risultati	7
6.1	Manuale d'uso	7
7	Conclusioni	8
7.1	Sviluppi futuri	8
	Glossario	I
	Bibliografia	III

Elenco delle figure

Elenco delle tabelle

2.1	Requisiti del progetto	3
-----	----------------------------------	---

Capitolo 1

Abstract

Le policy di sicurezza di rete moderne spesso isolano sia client che server rendendo impraticabili le connessioni dirette senza l'utilizzo di complesse configurazioni. Questo progetto espone il design e lo sviluppo di un'architettura client-server, pensata per superare le limitazioni imposte da firewall e NAT nelle reti private. Il meccanismo centrale sostituisce le tradizionali connessioni socket con un approccio basato su REST + WebSocket, utilizzando Appwrite¹ [1] come Proxy. Il sistema crea un canale di comunicazione sicuro su cui client e server interagiscono asincronamente tramite eventi di creazione di documenti nel database di Appwrite.

Viene quindi sviluppata una libreria modulare in C++ (con QT [2]) per astrarre l'API di Appwrite, implementando funzionalità come la gestione delle sessioni, l'identificazione dei canali basata su UUID e la serializzazione dei payload in JSON. L'implementazione di tale libreria viene validata attraverso lo sviluppo di una semplice applicazione Proof of Concept (PoC), un frontend, realizzato con Qt/QML e compilato con WebAssembly [3] per l'esecuzione in browser, e un servizio backend nativo in C++.

Modern security policies often restrict direct client-server communication, especially in private networks. This project details the design and development of a client-server architecture, designed to overcome the limitations of firewalls and NAT in private networks. The core mechanism replaces the traditional client-server socket connections with a REST + WebSocket based approach, using Appwrite as a Proxy. The system creates a secure communication channel on which clients and servers interact asynchronously via document creation events in the Appwrite-hosted database.

A modular C++ (with QT) library is developed to abstract the Appwrite API, implementing features such as session management, UUID-based channel identification and JSON payload serialization. The implementation of said library is validated through the development of a simple Proof of Concept (PoC) application, a frontend, built with Qt/QML and compiled with WebAssembly for browser-based execution, and a native C++ backend service.

¹<https://appwrite.io>

Capitolo 2

Introduzione

Il progetto nasce dall'esigenza di risolvere una delle sfide che più comunemente affligge le architetture distribuite moderne, ovvero la comunicazione bidirezionale asincrona tra un client ed un server che risiedono in reti private.

Lo scopo del progetto è quindi lo sviluppo di una libreria C++ che permetta di utilizzare Appwrite come middleware (proxy) per facilitare la creazione di canali di comunicazione sicuri e scalabili tra client e server. Al posto di tentare connessioni dirette instabili o poco sicure, entrambe le entità si connettono ad un'istanza Appwrite che funge da intermediario autenticato per lo scambio di messaggi strutturati in tempo reale. Il sistema deve garantire la sicurezza, fornendo accesso solamente ad utenti autenticati e autorizzati per l'accesso ai canali di comunicazione, delegando la gestione di credenziali, sicurezza e persistenza all'infrastruttura sottostante di Appwrite.

Gli obiettivi principali del lavoro sono articolati in diverse aree chiave:

- **Progettazione di un'architettura sicura:** è necessario definire un modello di comunicazione dove il server agisca da amministratore dell'infrastruttura di comunicazione (essendo autenticato tramite API Key) ed il frontend come utente finale (autenticandosi come utente "normale"). Il server è inoltre responsabile della gestione dei canali di comunicazione, fornendo un meccanismo di creazione, lettura, aggiornamento e cancellazione (CRUD) per canali, sessioni, mentre invece il frontend si preoccupa semplicemente di connettersi a canali esistenti, inviare messaggi e ricevere notifiche di nuovi messaggi.
- **Sviluppo della libreria appcom:** una libreria modulare, scritta in C++ e basata su Qt, che fornisca un'interfaccia semplice ed intuitiva per interagire con l'API di Appwrite è necessaria per la strutturazione del progetto; questa deve includere funzionalità per la gestione delle sessioni, l'identificazione dei canali basandosi su UUID, la serializzazione, validazione e impacchettamento dei payload e la gestione degli eventi di comunicazione in modo asincrono.
- **Proof of Concept:** per validare l'efficacia e il funzionamento corretto del sistema è necessario lo sviluppo di una semplice applicazione che dimostri la maniera di utilizzo della libreria in un contesto reale ed il funzionamento tecnico del meccanismo di comunicazione. Questa consiste in una chat semplice, dove un frontend, costruito con Qt/QML e compilato con WebAssembly per l'esecuzione tramite web browser, comunica con un servizio backend nativo in C++, attraverso i canali di comunicazione gestiti da Appwrite tramite appcom. Per semplificare la dimostrazione, il backend si limita al ruolo di *echo server*.
- **Bootstrap deploy:** per facilitare l'adozione dell'intermediario Appwrite in soluzioni esistenti, è necessario che, in presenza di un'istanza "pulita" di Appwrite ed il ser-

ver, quest'ultimo sia in grado, solamente tramite la chiave API e l'URL dell'istanza, di creare in autonomia l'infrastruttura necessaria per il funzionamento, dimodoché sia possibile integrare Appwrite sia come "primo cambiamento" in un'architettura esistente, sia nel caso si necessiti di scalare orizzontalmente un sistema già configurato con questo proxy, permettendo di cambiare istanza in modo dinamico, senza necessità di riconfigurazione.

2.1 Pianificazione

1. **Analisi:**
2. **Applicazione demo:**
3. **Definire struttura API:**

2.2 Requisiti

ID	Descrizione	Note
R-01	<i>Proxy</i> deve supportare l'API di Appwrite	-
R-02	X	
R-03	X	
R-04	X	
R-05	X	
R-06	X	
R-07	X	
R-08	X	
R-09	X	

Tabella 2.1: Requisiti del progetto

Capitolo 3

Stato dell'arte

Capitolo 4

Design e implementazione

4.1 API

Di seguito è riportato un esempio di codice per l'inizializzazione.

```
// Esempio di inizializzazione  
float x = 0.5f;  
std::vector<float> input = {x, x, x}; // Input per il modello  
// Altri parametri...
```

Capitolo 5

Test e validazione

5.1 Risultati

Capitolo 6

Risultati

6.1 Manuale d'uso

Capitolo 7

Conclusioni

7.1 Sviluppi futuri

Glossario

API Application Programming Interface. Un insieme di definizioni e protocolli utilizzati per costruire e integrare il software applicativo. 1

JSON JavaScript Object Notation. Un formato di testo standard aperto, leggero e leggibile dall'uomo, utilizzato per lo scambio di dati tra client e server. 1

NAT Network Address Translation. Una tecnica usata dai router per modificare gli indirizzi IP dei pacchetti in transito, permettendo a più dispositivi di condividere un singolo indirizzo IP pubblico. 1

Proxy Un componente software o hardware che funge da intermediario tra un client e un server, inoltrando le richieste e le relative risposte. 1

UUID Universally Unique Identifier. Un identificativo standardizzato a 128 bit utilizzato nei sistemi informatici per identificare in modo univoco informazioni e risorse. 1

WebAssembly Un formato di istruzioni binario portabile, progettato per consentire l'esecuzione di codice ad alte prestazioni all'interno dei browser web. 1

Bibliografia

- [1] Appwrite Team. *Appwrite Documentation - Open-Source Backend Server*. 2026. URL: <https://appwrite.io/docs> (visitato il 18/02/2026).
- [2] The Qt Company. *Qt 6 Documentation*. 2026. URL: <https://doc.qt.io/qt-6/> (visitato il 18/02/2026).
- [3] WebAssembly Community Group. *WebAssembly Specification*. 2026. URL: <https://webassembly.org/> (visitato il 18/02/2026).